

SIMULATED TUTORING RECORD

SWT281 Tutoring Session Transcript

White-box testing, black-box testing, real-world applications, and an introduction to automated testing with xUnit

Tutor	Lebo Nkosi
Student	Mosa Msiza
Module	SWT281 - Software Testing
Recorded duration	Approximately 57 minutes 42 seconds
Session format	Online one-to-one tutoring session
Booking summary	Help understanding white-box and black-box testing, their real-world use, and an introduction to automated testing with xUnit.

This is a fictional, purpose-built transcript created for testing the CampusLearn Tutor Head AI assessment workflow. It is not a transcript of an actual recorded conversation.

Opening and session goals

[00:00] **Lebo Nkosi:** Hi, Mosa. Welcome to the session. Before we begin, can you hear me clearly, and are you comfortable with us working through examples together?

[00:12] **Mosa Msiza:** Yes, I can hear you clearly. I am ready.

[00:18] **Lebo Nkosi:** Great. I reviewed your booking summary for SWT281. You would like help understanding white-box testing and black-box testing, how they are used in real projects, and an introduction to automated testing using xUnit. Is there anything else you want us to include?

[00:43] **Mosa Msiza:** That covers it. I know the definitions at a basic level, but when I see a scenario I struggle to decide which type of testing it is. I also have not written an xUnit test before.

[01:05] **Lebo Nkosi:** That gives us a useful goal. By the end, you should be able to compare the approaches, select suitable test cases for a real feature, and explain the structure of a basic xUnit test. We will pause for short knowledge checks along the way. Please interrupt me whenever something is unclear.

[01:34] **Mosa Msiza:** Okay, that sounds good.

[01:40] **Lebo Nkosi:** To establish a starting point, imagine you receive a login page but no source code. You enter usernames and passwords and compare the result with the requirements. What kind of testing do you think that is?

[02:02] **Mosa Msiza:** I think that is black-box testing because I cannot see what is happening inside the application.

[02:10] **Lebo Nkosi:** Exactly. That is the central idea, and we will now make it more precise.

Black-box testing

[02:25] **Lebo Nkosi:** Black-box testing evaluates externally visible behaviour. The tester supplies inputs, observes outputs, and compares them with requirements. Knowledge of the implementation is not required. The box could contain any code; the tester is concerned with whether it behaves correctly.

[03:02] **Mosa Msiza:** Does that mean black-box testing is always manual?

[03:08] **Lebo Nkosi:** No. Black-box describes the perspective, not whether the test is manual or automated. A person can manually test a login form, or an automated test can send an HTTP request and verify the response without inspecting the internal code. Both can be black-box tests.

[03:45] **Lebo Nkosi:** Consider a password-reset feature. Requirements might state that a registered email receives a reset link, an unknown email receives a neutral confirmation message, an empty email is rejected, and the reset token expires after fifteen minutes. A black-box tester derives cases from those rules.

[04:20] **Mosa Msiza:** Why should an unknown email receive a neutral message instead of saying the account does not exist?

[04:28] **Lebo Nkosi:** Good question. A neutral response prevents someone from using the form to discover which addresses have accounts. That is a security requirement. Testing is not only about happy paths; it also checks privacy, security, usability, and error handling.

[05:03] **Lebo Nkosi:** A common black-box technique is equivalence partitioning. We divide possible inputs into groups expected to behave similarly, then select representative values. If an age field accepts 18 through 65, the groups are below 18, 18 through 65, and above 65.

[05:37] **Mosa Msiza:** Then values like 17, 30, and 66 could represent the groups?

[05:43] **Lebo Nkosi:** Correct. We do not test every integer. We choose useful representatives. Another technique is boundary-value analysis. Defects commonly occur at limits because a developer may use less-than instead of less-than-or-equal-to.

[06:13] **Lebo Nkosi:** For the age field, boundary tests could include 17, 18, 19, 64, 65, and 66. These test just below, on, and just above each boundary.

[06:35] **Mosa Msiza:** Would zero and a very large number still be useful?

[06:42] **Lebo Nkosi:** Yes. They can reveal validation problems, but boundary values usually give the highest value first. Additional cases depend on risk. For a medical or financial system, we would normally test more aggressively than for a low-risk preference field.

[07:17] **Lebo Nkosi:** Other black-box techniques include decision tables for combinations of business rules, state-transition testing when behaviour depends on the current state, and use-case testing for complete user journeys.

[07:42] **Mosa Msiza:** Can you give an example of state-transition testing?

[07:48] **Lebo Nkosi:** Think of a bank card PIN. The card starts active. One incorrect attempt keeps it active, a second incorrect attempt still allows another try, and a third incorrect attempt blocks it. Tests must check the transitions between active, warning, and blocked states, including whether a correct PIN resets the failed-attempt count.

[08:27] **Mosa Msiza:** That makes sense because the same PIN input can produce a different result depending on previous attempts.

[08:36] **Lebo Nkosi:** Exactly. That is why understanding the state is important.

White-box testing

[08:52] **Lebo Nkosi:** White-box testing uses knowledge of the internal code structure. The tester designs cases around statements, branches, conditions, loops, exception paths, and interactions within the implementation.

[09:18] **Lebo Nkosi:** Suppose a method classifies a mark. It first rejects marks below zero or above one hundred. It returns Distinction for seventy-five and above, Pass for fifty through seventy-four, and Fail below fifty. By reading the code, we know there are validation and classification branches that should be exercised.

[09:52] **Mosa Msiza:** Would we test negative one, zero, forty-nine, fifty, seventy-four, seventy-five, one hundred, and one hundred and one?

[10:02] **Lebo Nkosi:** That is an excellent set. Notice that you used boundary thinking while also covering known branches. White-box and black-box techniques can complement each other; they are not enemies.

[10:27] **Lebo Nkosi:** Statement coverage asks whether every executable statement ran. Branch coverage asks whether every decision outcome ran. Condition coverage checks the true and false outcomes of individual Boolean conditions. Path coverage examines combinations of routes through the code, although complete path coverage can become impractical in complex programs.

[11:06] **Mosa Msiza:** If the test report shows one hundred percent coverage, does that prove there are no bugs?

[11:13] **Lebo Nkosi:** No. Coverage shows that code was executed, not that it was verified correctly. A test can call a method without making a meaningful assertion. Coverage also cannot prove that missing requirements were implemented. High coverage is useful evidence, but it is not proof of correctness.

[11:52] **Lebo Nkosi:** For example, a discount method might return fifty percent instead of five percent. A weak test that only checks the result is not null could execute every line and still miss the defect. The assertion must express the actual business expectation.

[12:20] **Mosa Msiza:** So a good test needs both useful input and a precise expected result.

[12:28] **Lebo Nkosi:** Correct. Test quality comes from the scenario, expected outcome, setup, and assertion, not from coverage alone.

[12:50] **Lebo Nkosi:** White-box testing is commonly performed by developers through unit tests. Testers can also use it when they have code access. Static analysis and code reviews are related structural quality activities, but they do not execute the program and therefore are not the same as dynamic white-box testing.

Comparing both approaches in a real system

[13:30] **Lebo Nkosi:** Let us apply both perspectives to an online store checkout. From a black-box perspective, what would you test?

[13:40] **Mosa Msiza:** I would test that valid items can be purchased, an expired card is rejected, the total is correct, and the customer receives confirmation.

[13:52] **Lebo Nkosi:** Good. We could also test an empty cart, an invalid delivery address, an unavailable item, discount-code rules, taxes, delivery fees, and what the customer sees when the payment provider is unavailable.

[14:25] **Lebo Nkosi:** From a white-box perspective, we might verify that the success and failure branches run, the correct payment-provider method is called, inventory is reduced only after payment succeeds, and a database transaction rolls back if an exception occurs.

[14:53] **Mosa Msiza:** Would checking that an email service was called also be white-box testing?

[15:00] **Lebo Nkosi:** Usually yes, because that test depends on knowledge of an internal collaboration. A customer only cares that the confirmation arrives. A unit test may verify that the checkout service called the notification interface once with the correct order details.

[15:32] **Lebo Nkosi:** Now consider a banking transfer. Black-box tests verify balances, validation messages, confirmation, permissions, and user-visible history. White-box tests target validation branches, transaction rollback, repository calls, audit logging, and exception handling.

[16:08] **Mosa Msiza:** Who normally writes each kind in a company?

[16:14] **Lebo Nkosi:** Developers often write unit and component tests with structural knowledge. Quality engineers design behavioural, integration, exploratory, and end-to-end tests. Business users may conduct acceptance testing. Strong teams share responsibility rather than treating quality as the tester's job alone.

[16:50] **Lebo Nkosi:** A useful test strategy has layers. Many fast unit tests protect business logic, fewer integration tests verify component boundaries, and a smaller number of end-to-end tests protect critical user journeys. This is often called the test pyramid.

[17:18] **Mosa Msiza:** Why should there be fewer end-to-end tests?

[17:23] **Lebo Nkosi:** They are slower, more expensive to maintain, and more likely to fail because of environment issues. They remain valuable for important workflows, but using them for every small rule makes feedback slow and fragile.

Automated testing fundamentals

[17:58] **Lebo Nkosi:** Automated testing means using code or tools to execute checks and compare actual outcomes with expected outcomes. Automation is useful for repeatable regression checks, rapid feedback, consistent execution, and continuous integration.

[18:30] **Mosa Msiza:** What exactly is a regression?

[18:35] **Lebo Nkosi:** A regression is when a change breaks behaviour that previously worked. For example, adding a new discount type might accidentally break the calculation for existing customers. A good automated test suite detects that before release.

[19:02] **Lebo Nkosi:** Not every test should be automated. Exploratory testing, visual judgement, and rapidly changing prototypes may benefit from human investigation. We automate stable, repeatable, high-value checks, particularly those run frequently.

[19:35] **Mosa Msiza:** What makes an automated test reliable?

[19:40] **Lebo Nkosi:** It should be deterministic, isolated where appropriate, fast enough for its level, readable, and independent of execution order. It should control dates, random values, network calls, and external dependencies instead of relying on unpredictable conditions.

[20:20] **Lebo Nkosi:** Tests should also clean up their data. A test that passes only when run first is a warning sign. Each test should arrange the state it needs and avoid leaking state into another test.

Introduction to xUnit

[20:48] **Lebo Nkosi:** xUnit is a popular .NET testing framework. A test project references the application project and usually follows a naming convention such as ApplicationName.Tests. The test runner discovers methods marked with xUnit attributes.

[21:15] **Lebo Nkosi:** The simplest attribute is Fact. It marks a test with one fixed setup. A basic calculator test creates the calculator, calls Add with two and three, and asserts that the result equals five.

[21:42] **Mosa Msiza:** Can we go through the structure line by line?

[21:47] **Lebo Nkosi:** Certainly. First, the Fact attribute tells xUnit this is a test. The method is public, returns void for synchronous code, and has a descriptive name. Inside it, we arrange the calculator, act by calling Add, and assert that the actual result equals the expected result.

[22:30] **Lebo Nkosi:** The pattern is called Arrange, Act, Assert. Arrange creates objects and input. Act performs the behaviour under test. Assert verifies the outcome. Keeping those phases visible makes tests easier to understand.

[23:00] **Mosa Msiza:** What happens when the expected value and actual value are different?

[23:06] **Lebo Nkosi:** The assertion throws a test failure. The runner identifies the failing test and normally displays expected and actual values. This feedback helps locate the defect or an incorrect test expectation.

xUnit example: Fact

[23:34] **Lebo Nkosi:** Here is the first complete example. Imagine the Calculator class already exists in the application project.

[24:05] **Lebo Nkosi:** The name Add_TwoPositiveNumbers_ReturnsTheirSum communicates the method, scenario, and expected outcome. Good names are valuable because the test report can explain failures without opening the source file.

[24:32] **Mosa Msiza:** Would it be wrong to call it TestOne?

[24:37] **Lebo Nkosi:** The test would still run, but the name would be unhelpful. Imagine a pipeline report containing hundreds of tests. Descriptive names reduce investigation time.

[25:02] **Lebo Nkosi:** We can also test negative values and zero. Repeating nearly identical methods is possible, but xUnit provides Theory for data-driven tests.

```
[Fact]
public void Add_TwoPositiveNumbers_ReturnsTheirSum()
{
    // Arrange
    var calculator = new Calculator();

    // Act
    int result = calculator.Add(2, 3);

    // Assert
    Assert.Equal(5, result);
}
```

xUnit example: Theory and InlineData

[25:28] **Lebo Nkosi:** A Theory runs the same test logic with multiple data rows. InlineData supplies values. We can provide two, three, and five; negative one, one, and zero; and zero, zero, and zero. xUnit treats each row as a separate test case.

[26:08] **Mosa Msiza:** If only one row fails, will it show which values caused the failure?

[26:14] **Lebo Nkosi:** Yes. Test explorers generally display the input values with the failed case. This makes a theory useful for equivalence classes and boundaries.

[26:38] **Lebo Nkosi:** Theory should be used when the behaviour and assertion remain the same across inputs. If a scenario requires a different setup or verifies a different rule, a separate test may communicate the intention more clearly.

[27:12] **Mosa Msiza:** So we should not force every test into one very large theory?

[27:18] **Lebo Nkosi:** Correct. Reducing duplicate code is useful, but readability is more important than making the fewest possible test methods.

```
[Theory]
[InlineData(2, 3, 5)]
[InlineData(-1, 1, 0)]
[InlineData(0, 0, 0)]
public void Add_DifferentNumbers_ReturnsExpectedSum(
    int first, int second, int expected)
{
    var calculator = new Calculator();
    int result = calculator.Add(first, second);
    Assert.Equal(expected, result);
}
```

SWT281 worked example: result classification

[27:42] **Lebo Nkosi:** Let us combine the testing techniques with xUnit. We have a ResultService.GetResult method. Marks outside zero through one hundred throw ArgumentOutOfRangeException. Marks below fifty return Fail, fifty through seventy-four return Pass, and seventy-five or higher return Distinction.

[28:20] **Lebo Nkosi:** Before looking at code, choose test values from the specification.

[28:28] **Mosa Msiza:** I would choose negative one, zero, forty-nine, fifty, seventy-four, seventy-five, one hundred, and one hundred and one.

[28:42] **Lebo Nkosi:** Excellent. Those values cover the lower validation boundary, fail-to-pass boundary, pass-to-distinction boundary, and upper validation boundary.

[29:05] **Lebo Nkosi:** For valid values, a Theory can pair each mark with the expected classification. For invalid values, another Theory can use Assert.Throws to verify the exception type.

[29:40] **Mosa Msiza:** How does Assert.Throws work with a method call?

[29:46] **Lebo Nkosi:** We pass a small function, often written as a lambda, that calls the method. xUnit executes it. The test passes only if the specified exception type is thrown. It fails when no exception or an unexpected exception is produced.

[30:22] **Lebo Nkosi:** If the method is asynchronous, xUnit provides `Assert.ThrowsAsync` and the test method returns `Task`. We await both the assertion and the application operation.

[30:48] **Mosa Msiza:** Would the invalid-mark tests be black-box or white-box?

[30:54] **Lebo Nkosi:** They can be black-box if derived from the public specification. If we added cases specifically to execute internal branches discovered by reading the code, that would be white-box. The same automated tool can support either perspective.

```
[Theory]
[InlineData(0, "Fail")]
[InlineData(49, "Fail")]
[InlineData(50, "Pass")]
[InlineData(74, "Pass")]
[InlineData(75, "Distinction")]
[InlineData(100, "Distinction")]
public void GetResult_ValidMark_ReturnsExpectedResult(
    int mark, string expected)
{
    var service = new ResultService();
    Assert.Equal(expected, service.GetResult(mark));
}

[Theory]
[InlineData(-1)]
[InlineData(101)]
public void GetResult_InvalidMark_ThrowsException(int mark)
{
    var service = new ResultService();
    Assert.Throws<ArgumentOutOfRangeException>(
        () => service.GetResult(mark));
}
```

Assertions and test quality

[31:32] **Lebo Nkosi:** Common xUnit assertions include `Equal`, `NotEqual`, `True`, `False`, `Null`, `NotNull`, `Contains`, `Empty`, `NotEmpty`, `Single`, `IsType`, and `Throws`. Choose the assertion that best communicates the expected behaviour.

[32:00] **Lebo Nkosi:** For a collection expected to contain exactly one item, `Assert.Single` is stronger and clearer than `Assert.True(items.Count greater than zero)`. Precise assertions catch more defects and explain intent.

[32:32] **Mosa Msiza:** Should a test have only one assertion?

[32:37] **Lebo Nkosi:** A test should focus on one behaviour. It may have several closely related assertions. For example, creating an order might reasonably verify the order status, total, and generated identifier. Many unrelated assertions usually signal that the test should be divided.

[33:10] **Lebo Nkosi:** Avoid testing private methods directly. Test observable behaviour through the public interface. If a private method contains complicated logic that is difficult to test, it may indicate that the logic deserves its own class.

[33:42] **Mosa Msiza:** What if the method uses the current date?

[33:47] **Lebo Nkosi:** Instead of calling the system clock directly throughout the code, inject a clock or `TimeProvider`. The test supplies a fixed time. This prevents tests from failing at midnight, month-end, or when run in another time zone.

Dependencies, mocks, and integration tests

[34:22] **Lebo Nkosi:** A unit test should isolate business logic from slow or unpredictable external systems. Suppose an `OrderService` depends on a payment gateway and email sender. We can supply test doubles that record calls and return controlled results.

[34:54] **Mosa Msiza:** Is that what a mock is?

[34:58] **Lebo Nkosi:** A mock is one kind of test double, usually used to verify interactions. A stub returns prepared values. A fake provides a lightweight working implementation, such as an in-memory repository. People sometimes use mock as a general term, but the distinctions can help when designing tests.

[35:35] **Lebo Nkosi:** For example, we could configure a payment stub to return success, call `Checkout`, assert that the order became paid, and verify that the notification service received the correct customer and order number.

[36:05] **Mosa Msiza:** Should we mock the database for every test?

[36:10] **Lebo Nkosi:** Not automatically. Mocking a database query can produce unrealistic behaviour. Unit-test pure business rules separately, then use integration tests with an appropriate test database to verify mappings, queries, constraints, and transactions.

[36:45] **Lebo Nkosi:** Integration tests check components working together. They are slower than unit tests but catch problems that mocks cannot, such as incorrect SQL, missing configuration, serialization issues, and dependency wiring.

[37:18] **Mosa Msiza:** We should never run tests against production, right?

[37:22] **Lebo Nkosi:** Correct. Automated tests must use controlled test environments and data. Production information must not be modified by a test suite.

Practical xUnit workflow

[37:48] **Lebo Nkosi:** A typical workflow is to create an xUnit test project, add a project reference to the application, create test classes, run dotnet test locally, and configure the same command in continuous integration.

[38:20] **Lebo Nkosi:** The test project usually includes the xUnit package, a test runner package, and the `Microsoft.NET.Test.Sdk` package. Development tools or project templates often configure these automatically.

[38:46] **Mosa Msiza:** When should tests run in a team project?

[38:51] **Lebo Nkosi:** Developers run focused tests while coding and the full relevant suite before committing. A continuous integration pipeline runs tests whenever code is pushed or a pull request is opened. Release pipelines may add integration, security, performance, and end-to-end checks.

[39:31] **Lebo Nkosi:** When a test fails, first read the failure message, expected value, actual value, and stack trace. Reproduce it in isolation, decide whether the application or test expectation is wrong, fix the cause, and run the wider suite to check for regressions.

[40:02] **Mosa Msiza:** Should we delete a test if it keeps failing randomly?

[40:07] **Lebo Nkosi:** No. A flaky test should be investigated. Common causes include shared state, timing assumptions, real network calls, uncontrolled randomness, and dependence on test order. Temporarily isolating it may be necessary, but silently deleting protection is risky.

Guided exercise: discount calculator

[40:45] **Lebo Nkosi:** Let us do a short exercise. A DiscountCalculator applies no discount below five hundred rand, ten percent from five hundred to nine hundred ninety-nine rand and ninety-nine cents, and fifteen percent from one thousand rand. Negative totals are invalid. Which cases would you choose?

[41:15] **Mosa Msiza:** Negative one for invalid, zero, four hundred ninety-nine rand and ninety-nine cents, five hundred, nine hundred ninety-nine rand and ninety-nine cents, and one thousand.

[41:28] **Lebo Nkosi:** Very good. Those are strong boundary cases. We might also add ordinary representatives such as two hundred, seven hundred fifty, and one thousand five hundred to show the normal behaviour inside each partition.

[41:52] **Lebo Nkosi:** Now state the expected discount for five hundred rand.

[41:58] **Mosa Msiza:** Ten percent, so the discount amount should be fifty rand.

[42:05] **Lebo Nkosi:** Correct. At one thousand rand, fifteen percent gives one hundred fifty rand. In a real monetary system, we would also pay attention to decimal types and rounding rules.

[42:34] **Lebo Nkosi:** How would you organize the xUnit tests?

[42:39] **Mosa Msiza:** A Theory for the valid totals and expected discounts, then a Fact or Theory using Assert.Throws for negative totals.

[42:50] **Lebo Nkosi:** Exactly. You could name the methods CalculateDiscount_ValidTotal_ReturnsExpectedAmount and CalculateDiscount_NegativeTotal_ThrowsArgumentOutOfRangeException.

[43:20] **Mosa Msiza:** Would testing four hundred ninety-nine rand and ninety-nine cents be black-box testing because it comes from the requirement boundary?

[43:29] **Lebo Nkosi:** Yes. If you then inspect the code and notice a special loyalty-customer branch that was not clearly specified, you would raise the requirement question and add structural tests after the expected behaviour is clarified.

Common testing mistakes

[44:02] **Lebo Nkosi:** Let us cover common mistakes. The first is testing implementation details too tightly. If a test fails whenever internal code is reorganized even though behaviour remains correct, it discourages safe refactoring.

[44:31] **Lebo Nkosi:** The second is using vague assertions. Checking only that a result is not null may miss incorrect values. The third is excessive setup. If every test requires dozens of unrelated objects, use builders or redesign the class responsibilities.

[45:02] **Mosa Msiza:** What about copying production data into tests?

[45:07] **Lebo Nkosi:** Avoid personal or confidential production data. Create synthetic test data that captures the required cases. Tests should be safe to share and repeat.

[45:32] **Lebo Nkosi:** Another mistake is relying only on happy paths. Useful suites include invalid input, boundaries, permissions, failures from dependencies, concurrency where relevant, and recovery behaviour.

[46:05] **Lebo Nkosi:** Finally, do not confuse passing tests with a finished product. Reviews, exploratory testing, accessibility checks, security testing, and monitoring still contribute to quality.

Knowledge check and correction

[46:35] **Lebo Nkosi:** I will give you short scenarios. Tell me the approach or concept. A tester validates a registration form using only the requirements and user interface.

[46:46] **Mosa Msiza:** Black-box testing.

[46:50] **Lebo Nkosi:** Correct. A developer reads a method and creates tests for every if and else outcome.

[47:00] **Mosa Msiza:** White-box testing and specifically branch coverage.

[47:05] **Lebo Nkosi:** Correct. A test runs the same rule with six different inputs.

[47:13] **Mosa Msiza:** An xUnit Theory with InlineData.

[47:18] **Lebo Nkosi:** Correct. A test expects invalid input to cause `ArgumentOutOfRangeException`.

[47:26] **Mosa Msiza:** Use `Assert.Throws`, or `Assert.ThrowsAsync` if the method is asynchronous.

[47:33] **Lebo Nkosi:** Excellent. One more: a suite executes every line, but its assertions only check that results are not null. Is that enough?

[47:44] **Mosa Msiza:** No. Coverage does not prove the results are correct. The assertions need to verify the business expectations.

[47:53] **Lebo Nkosi:** Exactly. Your explanations are accurate.

Student-led application

[48:10] **Lebo Nkosi:** Choose a feature from a campus learning system and explain how you would test it using both approaches.

[48:21] **Mosa Msiza:** I will use booking a tutor session. For black-box testing, I would check that a student can select an available tutor, module, time, and location, and that the booking confirmation shows the correct information.

[48:40] **Mosa Msiza:** I would also test unavailable time slots, missing fields, a location that does not match the tutoring mode, and two students trying to book the same slot.

[48:56] **Lebo Nkosi:** Good. You included normal, validation, and concurrency-related scenarios. What about white-box testing?

[49:06] **Mosa Msiza:** I would inspect the booking service branches, test the availability check, verify that the booking is saved only once, check that the notification is created, and test the branch that releases an expired reservation.

[49:24] **Lebo Nkosi:** Excellent. What would you unit-test and what would you integration-test?

[49:32] **Mosa Msiza:** I would unit-test rules such as whether the location matches online or face-to-face tutoring. I would integration-test saving the booking, unique database constraints, and perhaps the API endpoint with the database.

[49:55] **Lebo Nkosi:** That is a strong distinction. For an end-to-end test, you could automate the critical path from signing in through receiving booking confirmation, while keeping the number of those tests manageable.

Reviewing an example test

[50:28] **Lebo Nkosi:** Suppose a test is named `BookingTest` and it creates a booking, calls the service, and checks only `Assert.NotNull`. What improvements would you make?

[50:41] **Mosa Msiza:** I would give it a descriptive name, use a clear Arrange, Act, Assert structure, and verify specific results such as the booking status, tutor, module, scheduled time, and maybe the saved record.

[50:58] **Lebo Nkosi:** Good. We should be careful not to turn one unit test into a broad integration test, but your expected outcomes are much more meaningful. We would select assertions according to the test level.

[51:25] **Lebo Nkosi:** If the rule says a student cannot book an occupied slot, a focused test name could be `CreateBooking_OccupiedSlot_ReturnsConflict`. The arrange step creates an occupied slot, the act step requests the same slot, and the assert step verifies the conflict result and that no second booking was stored.

[51:59] **Mosa Msiza:** That also makes the requirement obvious when someone reads the test.

[52:05] **Lebo Nkosi:** Exactly. Tests can serve as executable documentation when they are clear.

Final recap and next steps

[52:25] **Lebo Nkosi:** Let us recap the session. Black-box testing derives checks from expected external behaviour and does not require implementation knowledge. White-box testing uses internal structure to target statements, branches, conditions, paths, and interactions.

[52:55] **Lebo Nkosi:** Equivalence partitioning reduces a large input space into representative groups. Boundary-value analysis focuses on edges. Decision tables handle combinations of rules, and state-transition testing checks behaviour that depends on previous state.

[53:25] **Lebo Nkosi:** Automated testing provides repeatable feedback and regression protection. In xUnit, Fact handles a fixed case, Theory handles multiple data rows, and assertions state the expected outcome. Arrange, Act, Assert gives tests a readable structure.

[53:58] **Lebo Nkosi:** Unit tests protect isolated logic, integration tests verify components working together, and a small set of end-to-end tests protects critical journeys. Coverage helps locate untested code, but meaningful scenarios and assertions determine quality.

[54:28] **Lebo Nkosi:** In your own words, what is the most important difference between white-box and black-box testing?

[54:38] **Mosa Msiza:** Black-box testing checks whether the system behaves according to requirements from the outside. White-box testing uses knowledge of the code to verify its internal logic and paths.

[54:52] **Lebo Nkosi:** Good. When would you use Theory?

[54:57] **Mosa Msiza:** When the same behaviour should be tested using several input and expected-output combinations.

[55:07] **Lebo Nkosi:** Correct. Your practice task is to implement the DiscountCalculator example and create at least one Fact, one Theory with boundary cases, and one exception test. Add brief comments identifying Arrange, Act, and Assert for your first version.

[55:40] **Mosa Msiza:** I will do that. I understand the difference much better now, especially that both approaches can be automated and used together.

[55:53] **Lebo Nkosi:** Excellent. Do you have any final question before we close?

[56:00] **Mosa Msiza:** Just one. Should I aim for one hundred percent coverage in the exercise?

[56:08] **Lebo Nkosi:** Aim to cover the stated rules and important boundaries first. Review the coverage report afterward to identify anything unintentionally missed. Do not add weak tests only to increase the percentage.

[56:38] **Mosa Msiza:** Understood. Thank you for the session.

[56:44] **Lebo Nkosi:** You are welcome. You participated well and selected the boundary cases accurately. Bring your xUnit test project to the next session if you would like feedback on the structure.

[57:05] **Mosa Msiza:** I will. Goodbye.

[57:10] **Lebo Nkosi:** Goodbye, Mosa. Have a good day.

[57:16] **Session note:** The call remained open briefly while both participants confirmed that the shared material had been saved. Recording ended at approximately 57 minutes and 42 seconds.